

ServiceNow Incident Form: Implement Client Script & UI Policy (Incident)

Team ID: SWTID-2026-7108

Project Description:

This project enhances the ServiceNow Incident form by implementing Client Scripts and UI Policies that make the form dynamic, guided and error-resistant. UI Policies control field behaviour (mandatory, visible, read-only) based on conditions such as State, Impact, Urgency and Category, while Client Scripts run in the browser to validate input, show helpful messages and react instantly when a user changes a field value.

The solution is built entirely on the ServiceNow Now Platform using a Personal Developer Instance (PDI). Three UI Policies enforce data-completeness rules (for example, Close code and Close notes when an incident is resolved), and four Client Scripts (onLoad, onChange and onSubmit types) guide the caller and the fulfiller while filling in the incident. The result is better data quality, fewer incomplete tickets and a smoother experience for service desk agents, without any server-side coding.

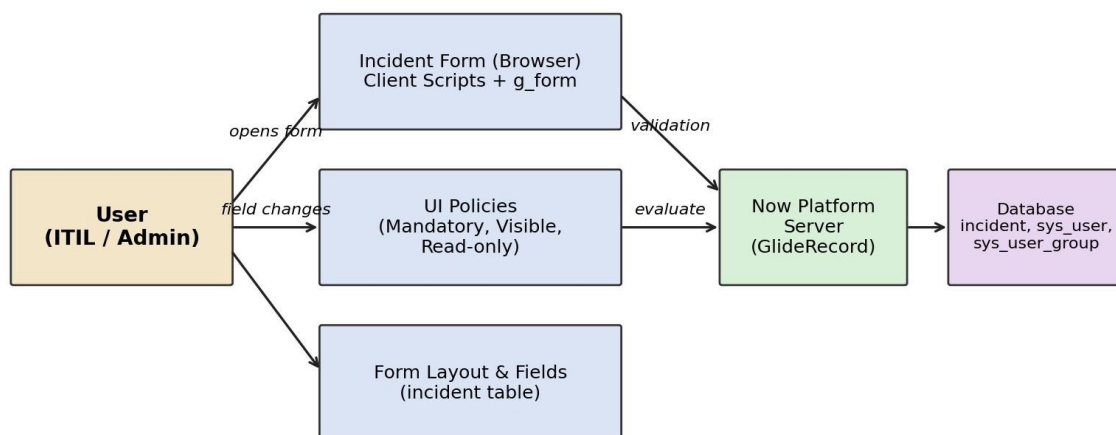
Scenarios

Scenario 1: A service desk agent opens a new incident. An onLoad Client Script displays an information message reminding the agent to record a clear short description. When the agent sets Impact and Urgency both to High, a UI Policy makes Assignment group and Description mandatory so that a critical incident is never saved without an owner and details.

Scenario 2: A fulfiller changes the State to Resolved. A UI Policy immediately makes Close code and Close notes visible and mandatory. If the fulfiller tries to save without them, the form highlights the missing fields and blocks the save.

Scenario 3: A user selects Impact = 1 - High. An onChange Client Script shows an inline field message that high-impact incidents are escalated. When the user submits a record with a very short Short description, an onSubmit Client Script stops the submission and asks for a more meaningful summary.

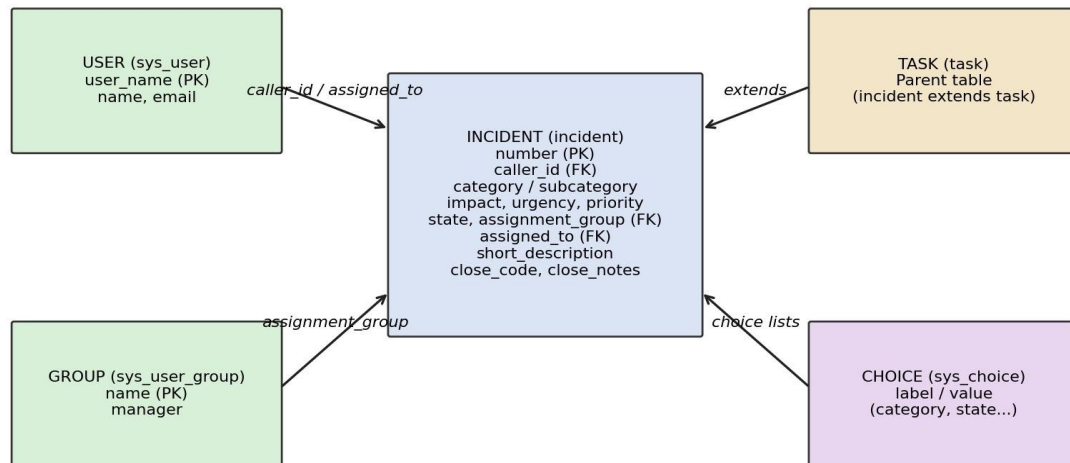
Technical Architecture:



Client-side logic runs in the browser; the server saves validated data to the incident table.

Description: The solution follows the standard three-tier ServiceNow architecture. The presentation tier is the Incident form in the browser, where Client Scripts (using the `g_form` and `g_user` APIs) and UI Policies execute. The application tier is the Now Platform server, which evaluates and saves the record and provides the choice lists and reference data. The data tier is the platform database, where the incident table stores the records. UI Policies and Client Scripts are client-side, so feedback to the user is instant and no extra server round-trip is needed for basic validation.

ER Diagram (Database Description)



The Incident table extends the Task table and references other platform tables. The main relationships used in this project are listed below.

Field on Incident	Referenced table	Purpose
caller_id	User (sys_user)	Person who reported the incident
assigned_to	User (sys_user)	Fulfiller working on the incident
assignment_group	Group (sys_user_group)	Team responsible for resolution
category / state / impact / urgency	Choice list (sys_choice)	Drive the conditions used by UI Policies and Client Scripts

Pre-requisites:

- **ServiceNow Developer Account:** Sign up at developer.servicenow.com and request a Personal Developer Instance (PDI).
- **ServiceNow Platform Basics:** Navigation, forms, lists, tables and the application navigator.
- **Client Script Concepts:** Types (onLoad, onChange, onSubmit, onCellEdit) and the `g_form` / `g_user` client APIs.
- **UI Policy Concepts:** Conditions, UI Policy Actions, scripts (Execute if true / false) and Reverse if false.
- **JavaScript Fundamentals:** Functions, conditions and string handling used in client scripts.
- **Incident Management Knowledge:** Incident fields such as Category, Impact, Urgency, Priority and State.
- **Update Sets:** Capturing and moving configuration changes between instances.

Project Workflow:

Milestone 1: Environment Setup and Requirement Analysis

- **Activity 1.1:** Sign up and provision a ServiceNow Personal Developer Instance.
- **Activity 1.2:** Explore the Incident table, form layout and fields.
- **Activity 1.3:** Define the business requirements and the UI Policy / Client Script design.
- **Activity 1.4:** Create a dedicated Update Set to capture all changes.

Milestone 2: UI Policy Implementation

- **Activity 2.1:** Create UI Policy for High Impact and High Urgency incidents.
- **Activity 2.2:** Create UI Policy for Resolved / Closed state.
- **Activity 2.3:** Create UI Policy for On Hold state.

Milestone 3: Client Script Implementation

- **Activity 3.1:** Create onLoad Client Script for new incidents.
- **Activity 3.2:** Create onChange Client Scripts for Impact and Category.
- **Activity 3.3:** Create onSubmit Client Script for Short description validation.

Milestone 4: Testing and Validation

- **Activity 4.1:** Prepare and execute test cases for each UI Policy and Client Script.
- **Activity 4.2:** Verify behaviour on new and existing incident records.

Milestone 5: Deployment

- **Activity 5.1:** Complete and export the Update Set.
- **Activity 5.2:** Import, preview and commit the Update Set on the target instance.

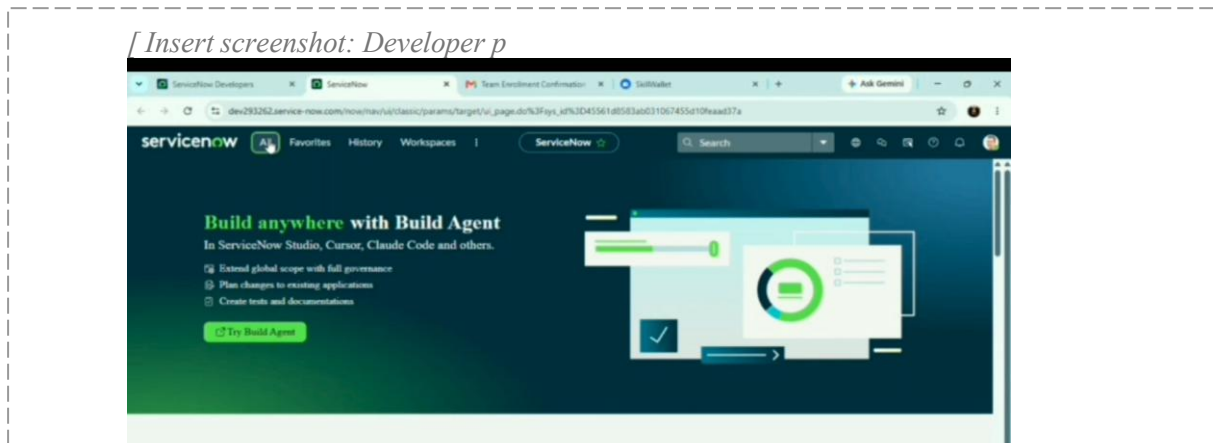
Milestone 6: Conclusion

Milestone 1: Environment Setup and Requirement Analysis

In this milestone the development environment is prepared, the Incident form is studied and the requirements for the UI Policies and Client Scripts are finalised.

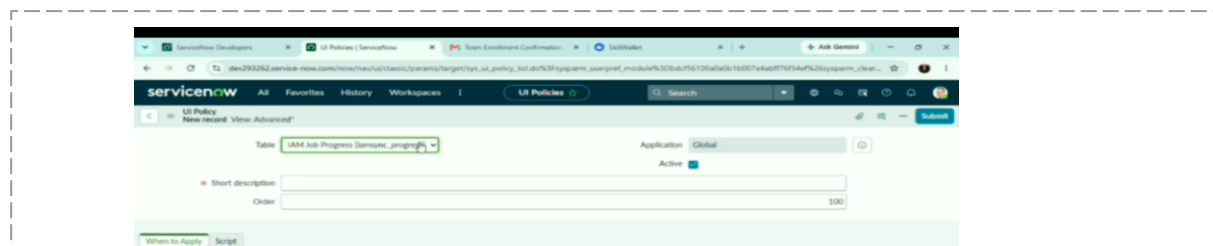
Activity 1.1: Provision a ServiceNow Personal Developer Instance

- **Step 1 — Create a developer account:** Visit <https://developer.servicenow.com> and sign up or log in with your ServiceNow ID.
- **Step 2 — Request an instance:** Click Request Instance and choose the latest available release. Wait until the instance is ready.
- **Step 3 — Log in:** Open the instance URL (<https://devXXXXX.service-now.com>) and log in as admin using the credentials shown in the developer portal.



Activity 1.2: Explore the Incident Table and Form

- **Open the form:** In the application navigator search Incident and open Create New to see the form.
- **Identify fields:** Note the key fields: Number, Caller, Category, Subcategory, Contact type, State, Impact, Urgency, Priority, Assignment group, Assigned to, Short description, Description, Close code and Close notes.
- **Inspect the table:** Navigate to System Definition > Tables and open the incident table to view its columns and its parent table (Task).



Activity 1.3: Define Requirements

S.No	Requirement	Solution used
1	Incidents with Impact = High and Urgency = High must always have an Assignment group and a Description.	UI Policy: Critical Incident Mandatory Fields
2	Resolving or closing an incident must capture Close code and Close notes.	UI Policy: Resolution Details Mandatory
3	Putting an incident On Hold must capture the reason.	UI Policy: On Hold Reason Mandatory

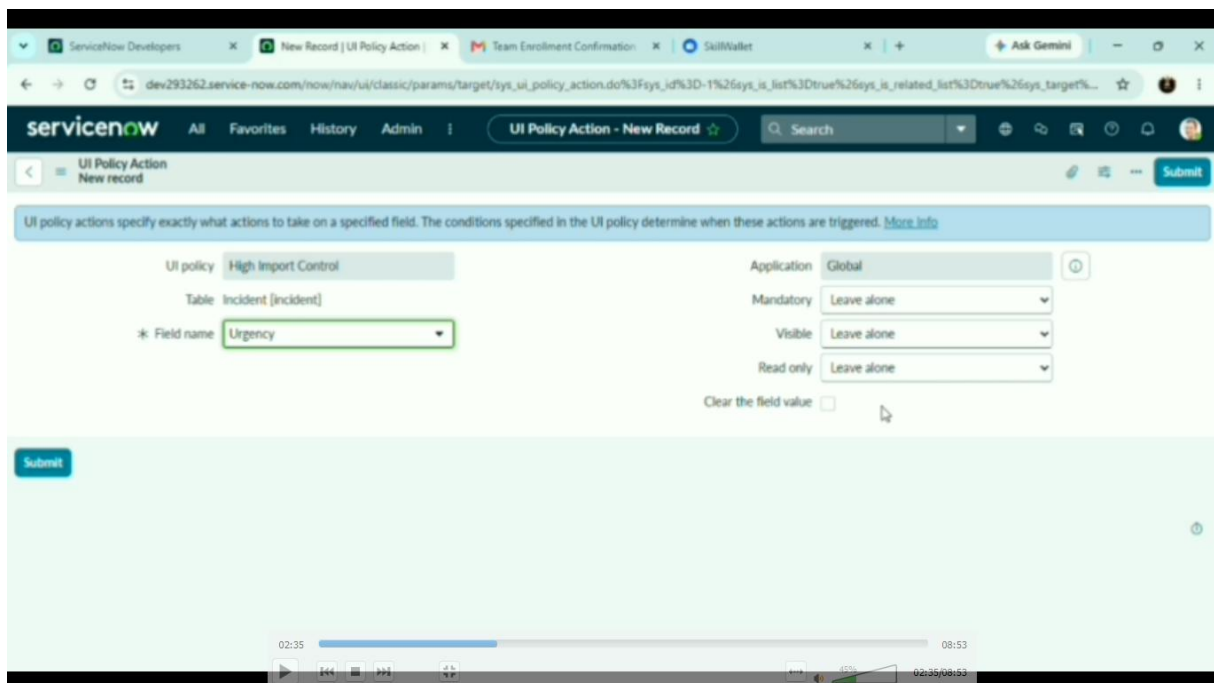
4	Agents are reminded to enter good details when	Client Script (onLoad): Incident New
---	--	--------------------------------------

S.No	Requirement	Solution used
	creating a new incident.	Record Guidance
5	Users are warned when a High impact is chosen.	Client Script (onChange): Impact High Warning
6	Category change should guide the user and reset dependent data.	Client Script (onChange): Category Change Handler
7	Short description must be meaningful before saving.	Client Script (onSubmit): Validate Short Description

Activity 1.4: Create an Update Set

- **Step 1:** Navigate to System Update Sets > Local Update Sets and click New.
- **Step 2:** Name it "Incident Client Script and UI Policy", set the state to In progress and click Submit.
- **Step 3:** Click Make this my current set so that every change made in the next milestones is captured.

[Insert screenshot: Update Set record created and set as current]



Milestone 2: UI Policy Implementation

UI Policies dynamically change the behaviour of form fields (Mandatory, Visible, Read-only) when a condition is met. They run on the client and do not require scripting for basic rules.

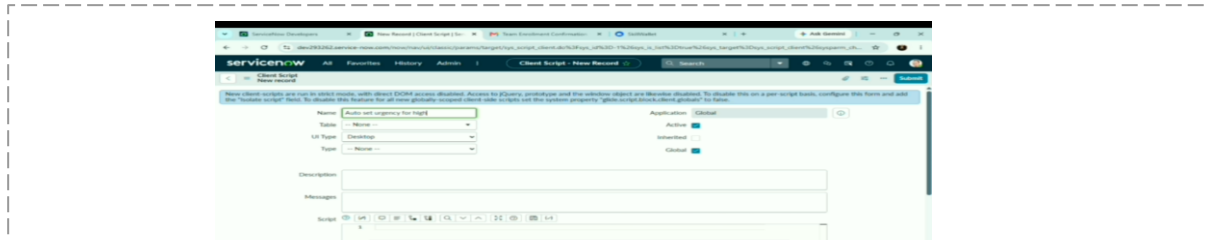
Activity 2.1: UI Policy — Critical Incident Mandatory Fields

- Step 1:** Navigate to System UI > UI Policies and click New.

Field	Value
Table	Incident [incident]
Short description	Critical Incident Mandatory Fields
Active	true
Conditions	Impact is 1 - High AND Urgency is 1 - High
Reverse if false	true
On load	true

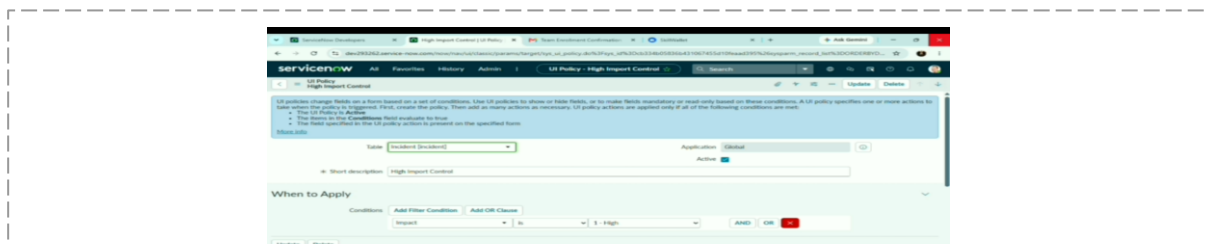
- Step 2:** Save the record. In the UI Policy Actions related list click New for each field below.

Field name	Mandatory	Visible	Read only
Assignment group	True	Leave alone	Leave alone
Description	True	Leave alone	Leave alone



Activity 2.2: UI Policy — Resolution Details Mandatory

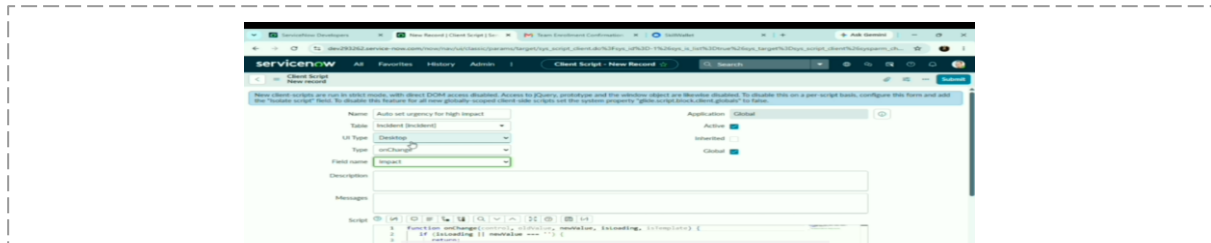
- Step 1:** Create a new UI Policy on the Incident table with the short description "Resolution Details Mandatory".
- Step 2:** Condition: State is Resolved OR State is Closed. Enable Reverse if false and On load.
- Step 3:** UI Policy Actions: Close code — Mandatory True, Visible True; Close notes — Mandatory True, Visible True.



Activity 2.3: UI Policy — On Hold Reason Mandatory

- **Step 1:** Create a UI Policy with the short description "On Hold Reason Mandatory".
- **Step 2:** Condition: State is On Hold. Enable Reverse if false and On load.
- **Step 3:** UI Policy Action: On hold reason (hold_reason) — Mandatory True, Visible True.

Note: field names and choice values can differ slightly between ServiceNow releases. Use the field picker in the UI Policy Actions to select the correct field on your instance.



Milestone 3: Client Script Implementation

Client Scripts are JavaScript functions that run in the user's browser when a form loads, a field changes or a form is submitted. They use the `g_form` API to read and change the form and the `g_user` API to read details of the logged-in user.

Activity 3.1: onLoad Client Script — Incident New Record Guidance

- **Step 1:** Navigate to System Definition > Client Scripts and click New.

Field	Value
Name	Incident New Record Guidance
Table	Incident [incident]
UI Type	All
Type	onLoad
Active	true

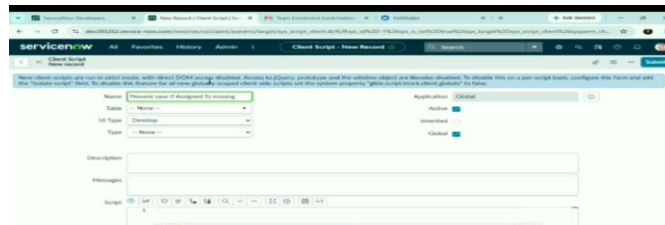
- **Step 2:** Paste the following script and click Submit.

```
function onLoad() {
    // Run only when a new incident is being created
    if (g_form.isNewRecord()) {
        g_form.addInfoMessage('Please enter a clear Short description and
select the correct Category, Impact and Urgency.');
```

```
        g_form.setValue('contact_type', 'phone');
```

```
    }
```

```
}
```



Activity 3.2: onChange Client Scripts

Impact High Warning

Type: onChange | Field name: Impact | Table: Incident

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    g_form.hideFieldMsg('impact');
    if (newValue == '1') {
        g_form.showFieldMsg('impact', 'High impact incidents are escalated to
the Major Incident team.', 'info');
```

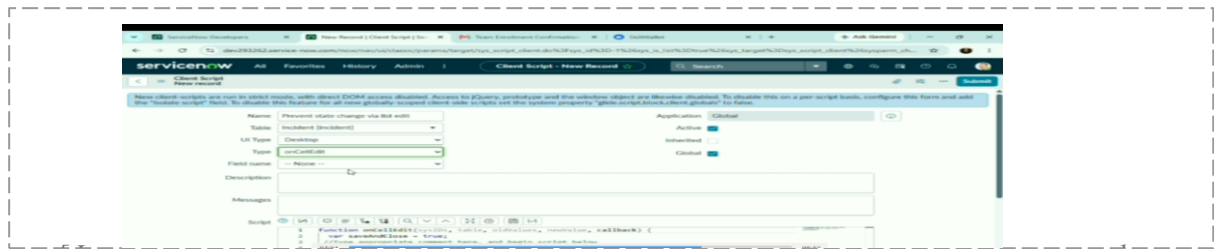
```
    }
```

```
}
```

Category Change Handler

Type: onChange | Field name: Category | Table: Incident

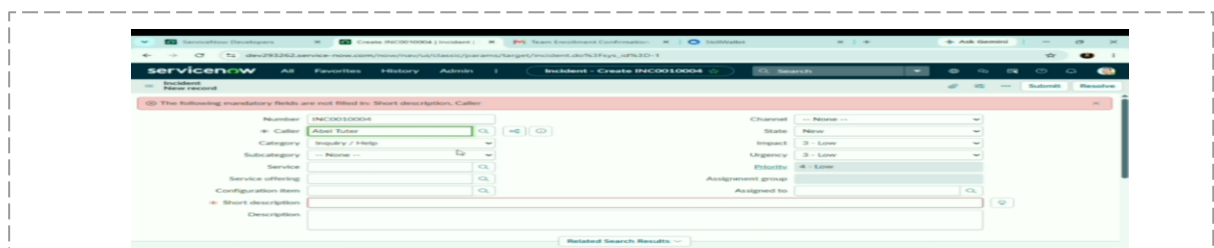
```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }
    g_form.clearMessages();
    if (newValue == 'hardware') {
        g_form.addInfoMessage('Hardware incident: please mention the device model and location in the Description.');
```



Activity 3.3: onSubmit Client Script — Validate Short Description

Type: onSubmit | Table: Incident

```
function onSubmit() {
    var shortDesc = g_form.getValue('short_description').trim();
    if (shortDesc.length < 10) {
        g_form.showFieldMsg('short_description', 'Short description must be at least 10 characters.', 'error');
        return false; // stop the submission
    }
    return true;
}
```



Note: g_form.getValue() returns a string, so trim() can be used directly. Client Scripts run only in the browser UI; for rules that must also apply to integrations and imports, add a server-side Business Rule or Data Policy.

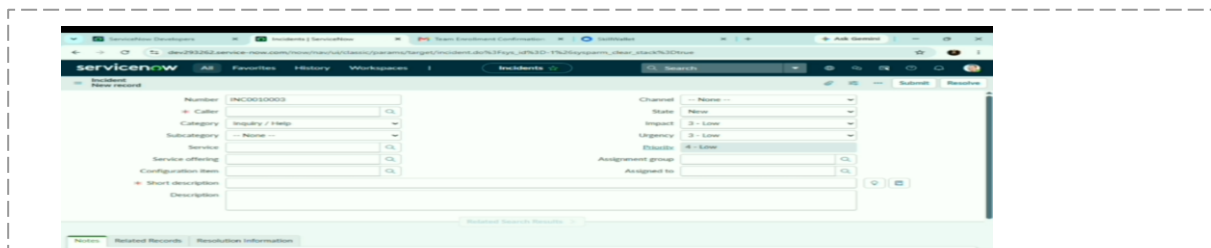
Milestone 4: Testing and Validation

Each UI Policy and Client Script is tested from the Incident form (Incident > Create New and by opening existing incidents). Expected and actual results are recorded below.

Activity 4.1: Test Cases

TC	Test scenario	Expected result	Script / Policy	Status
1	Open Incident > Create New	Info message is displayed; Contact type is Phone	onLoad script	Pass / Fail
2	Set Impact = 1 - High and Urgency = 1 - High	Assignment group and Description become mandatory	UI Policy 1	Pass / Fail
3	Change Urgency back to a lower value	Mandatory rule is removed (Reverse if false)	UI Policy 1	Pass / Fail
4	Set State = Resolved and save without close details	Close code and Close notes are mandatory; save blocked	UI Policy 2	Pass / Fail
5	Set State = On Hold	On hold reason becomes mandatory	UI Policy 3	Pass / Fail
6	Select Impact = 1 - High	Field message about escalation appears on Impact	onChange (Impact)	Pass / Fail
7	Select Category = Hardware	Info message asking for device details appears	onChange (Category)	Pass / Fail
8	Save with Short description of 5 characters	Error shown; record is not saved	onSubmit script	Pass / Fail
9	Save with a valid Short description	Incident saves successfully	onSubmit script	Pass / Fail

Tip: replace Pass / Fail with the actual result after executing each test case and attach screenshots.



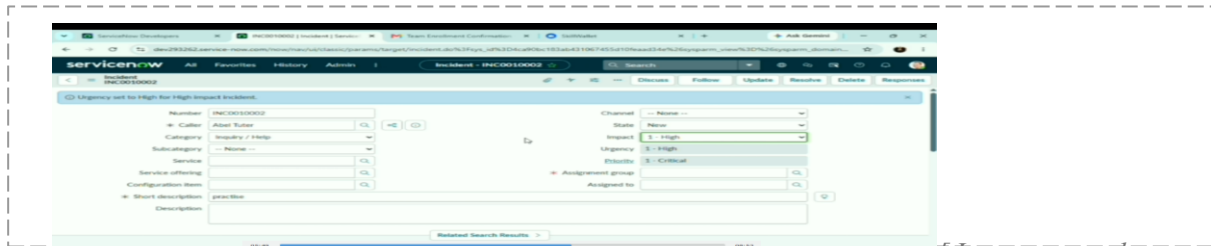
Activity 4.2: Debugging

- **Console:** Use the browser developer console (F12) to check for JavaScript errors in Client Scripts.
- **Field Watcher:** Right-click a field label on the Incident form and choose Watch field to see which UI Policy or Client Script changed its value, mandatory or visible state (admin role required).
- **Checklist:** Confirm the script is Active, the UI Type is correct and the table is Incident.

Milestone 5: Deployment

Activity 5.1: Complete and Export the Update Set

- **Step 1:** Open System Update Sets > Local Update Sets and select the update set created in Milestone 1.
- **Step 2:** Verify that the Customer Updates related list contains all three UI Policies (with their actions) and all Client Scripts.
- **Step 3:** Set State to Complete and use Export to XML to download the update set file.



Activity 5.2: Deploy to a Target Instance

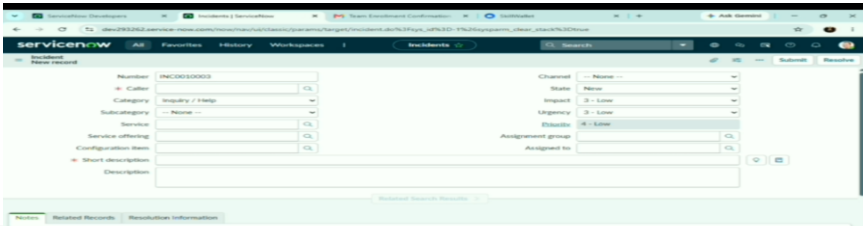
- **Step 1:** Log in to the target instance and open System Update Sets > Retrieved Update Sets.
- **Step 2:** Click Import Update Set from XML, choose the exported file and upload it.
- **Step 3:** Open the retrieved update set, click Preview Update Set and resolve any problems.
- **Step 4:** Click Commit Update Set. Open a new incident to confirm the behaviour.

The screenshot displays the ServiceNow interface for the 'UI Policies' table. The 'Running transactions' list is shown, with the following data:

Short description	Table	Conditions	Reverses If False	On load	Updated	Order
UI Policy to hide related features Tags when field expansion and so	UI (L4) Sys_In_Rpt	role_ugrations=ui"UI	True	True	2025-05-04 14:55:01	0
Hide fields on mandatory without discovery	Security Tables (sys_security_data)		False	True	2025-05-08 14:58:47	0
Always Hide Read-only Related	Relationships (sys_relationships)	rel_DISPLAY"UI	True	True	2025-05-09 12:29:53	1
Hide extra fields	Local Sys. (sys_local)		True	True	2025-05-09 12:40:37	1
Set Tables column Read-Only	To the Widget Mapping (sys_ui_widget_mapping)		True	True	2025-05-09 13:35:33	1
Clear and generate table and insertion-allow when permissions are not required	Function (sys_ui_function)	permissions=signature"UI	True	True	2025-05-18 07:59:13	1
Content on parameters	HTTP Method (sys_http_method)	url_method=GET,method=delete"UI	True	True	2025-05-18 14:54:13	1
Instance Class and Change Profile	System Class (sys_instance)	profile=SYSTEM"UI	True	True	2025-05-21 11:44:25	1

Exploring the Website's Web Pages:

Incident Form:

A screenshot of the ServiceNow Incident form. The form is titled 'Incident' and shows various fields for creating a new incident. Fields include 'Number' (INC0000000), 'Category' (Incident / Help), 'Subcategory' (None), 'Service' (None), 'Service offering' (None), 'Configuration item' (None), 'Short description' (None), 'Channel' (None), 'State' (New), 'Impact' (1 - Low), 'Urgency' (1 - Low), 'Escalate' (None), 'Assignment group' (None), and 'Assignment to' (None). The form is displayed in a web browser window.

Description: The standard Incident form is enhanced with client-side logic. When a new record opens, guidance is shown at the top of the form and Contact type is pre-filled. Fields react instantly as values change.

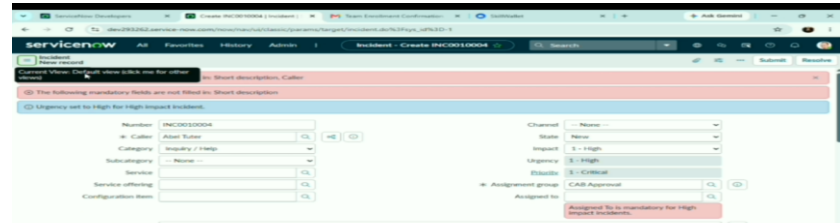
Mandatory Fields in Action:

[Insert screenshot: Incident with State = Resolved showing mandatory Close code and Close notes

A screenshot of the ServiceNow Incident form with the state set to 'Resolved'. The form shows mandatory fields for 'Close code' and 'Close notes'. The 'Close code' field is labeled 'Close code' and the 'Close notes' field is labeled 'Close notes'. The form is displayed in a web browser window.

Description: When the State is set to Resolved or Closed, Close code and Close notes turn mandatory. Similarly, High impact with High urgency makes Assignment group and Description mandatory, and On Hold makes On hold reason mandatory.

Validation Message:

A screenshot of the ServiceNow Incident form showing a validation message. The message is displayed in a red box at the top of the form and reads: 'The following mandatory fields are not filled in: Short description'. The form is displayed in a web browser window.

Description: The onSubmit script blocks the save and displays an error message on the Short description field when fewer than 10 characters are entered.

Conclusion:

This project demonstrates how Client Scripts and UI Policies improve the ServiceNow Incident form without server-side code. UI Policies enforce data completeness by making Close code, Close notes, Assignment group, Description and On hold reason mandatory at the right moment, while Client Scripts guide users on load, respond to field changes and validate input on submit. The configuration was built in a Personal Developer Instance, captured in an Update Set and deployed by import and commit. Together these controls raise incident data quality, reduce rework for resolver groups and give service desk agents immediate feedback. Future enhancements can include Catalog Client Scripts for service requests, server-side Business Rules or Data Policies to enforce the same rules for integrations, and dashboards to measure improvements in incident data quality.